INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR
END-SPRING SEMESTER EXAMINATION 2023-24

Subject No.: CS60075 **Subject: Natural Language Processing**

Duration: 3 Hours **Full Marks: 70**

Department: Computer Science and Engineering

Specific charts, graph paper, log book etc., required: NA

Special Instructions (if any):

- *Attempt all questions. All parts of the same question must be answered together.*
- *Answers should be brief and to-the-point. Sketchy answers will be penalized.*
- *Make reasonable assumptions only if necessary, and state any assumptions made. No clarification can be provided in the examination hall.*
- *All working steps must be shown. Writing the answer without showing steps will not be given any credit, even if the answer is correct. You can use calculators.*

Question 1 (short answer questions) [20]

Answer the following questions briefly and to-the-point.

(a) What is the k-th order Markov assumption in the context of a language model? What is the practical utility of the Markov assumption? [2]

(b) What metric will you use for intrinsic evaluation of a Language Model? State the mathematical expression of the metric. [2]

(c) What is "teacher forcing"? Is this technique used during training or evaluation of NLP models? [2]

(d) According to the paper "Attention Is All You Need", state two primary motivations behind the self-attention mechanism. [2]

(e) How can one decide the number of neurons in each layer of a Word2vec skip-gram neural network? [3]

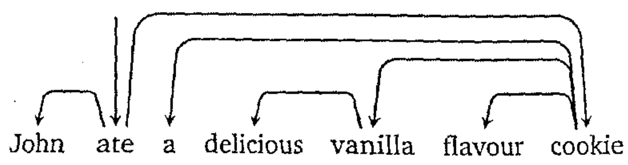
(f) State the conditions for a directed graph to be considered a dependency graph. [3]

(g) State the three distinct types of attention mechanism used in the Transformer architecture. Your answer should clearly bring out the differences among the types. [3]

(h) Write 2-3 sentences on how you would use a BERT encoder model to do each of the following NLP tasks: (i) Part of Speech Tagging (ii) Spam Email Classification. [3]

Question 2 (Dependency graphs, parsing) [7 + 3 = 10]

(a) Consider the dependency parse (un-labeled) given below. Using the standard Arc-Eager parsing technique, obtain the sequence of transitions (Shift, Reduce, Left-Arc, Right-Arc) to generate the tree shown.



(b) During dependency parsing, we can represent a configuration C (consisting of the current contents of the stack S, buffer B and already decoded arcs A) as a 4-dimensional boolean feature vector $f(C) = [f_1, f_2, f_3, f_4]$.

Suppose these are designed as:

(i) f_1: 1 if stack is empty, 0 otherwise

(ii) f_2: 1 if $top(S)$ is NOUN **AND** $first(B)$ is VERB, 0 otherwise

(iii) f_3: 1 if $top(S)$ is PREPOSITION, 0 otherwise

(iv) f_4: 1 if $top(S)$ ∈ {'John', 'cookie'} **OR** $first(B)$ ∈ {'delicious', 'vanilla'}, 0 otherwise

Using the features listed above, find $f(C)$ for the <u>*first six configurations*</u> (including starting configuration) while parsing the sentence given in Q2(a).

Question 3 (Word embeddings) [6 + (3+1) = 10]

(a) Suppose you are learning the word vectors of your vocabulary using the SkipGram Word2vec model. Assume there are only 7 words in your vocabulary {v1, v2, v3, v4, v5, v6, v7}. At a certain time-step during the training, the sliding window is {v5, v3, v6}, with v3 as the center word and <u>both</u> v5 and v6 as the context words. Also assume that all the seven words have 4-dimensional in and out vectors, which have the same values at this time-step given by (in the order v1, v2, ..., v7): [1, −1, 2, 0], [−1, -2, 0, 1], [1, 1, 2, -2], [1, −2, 0, 1], [1, 0, -1, 1], [1, 1, 0, -1], [0, 1, 1, -2]. Draw a diagram depicting the values at the input, hidden and output layers at this time-step. What will be the total loss for the given window? Use element-wise dot product for matrix calculations and Softmax as the activation function in the output layer.

(b) Suppose you have a small corpus "the brown fox", so that these are the only three words in your vocabulary. Assume that the input encodings for the three words are [1,0,0] for 'the', [0,1,0] for 'brown' and [0,0,1] for 'fox'. In a CBOW architecture, after training is done, the transformation matrices from the input layer to the hidden layer and from the hidden layer to the output layer are respectively:

$$\begin{bmatrix} 0.1 & 0.4 \\ 0.2 & 0.5 \\ 0.4 & 0.3 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.5 & 0.3 & 0.1 \end{bmatrix}$$

If the context words are "the" and "fox", calculate the representations at the input, hidden and output layers. If the model needs to be trained for 2 more epochs, how many learnable parameters need to be updated in total and why? Ignore the bias terms.

Question 4 (RNNs) [(1 + 2 + 1 + 2) + 4 = 10]

(a) Consider a Named-Entity Recognition (NER) task where the goal is to classify each word in a sentence as either a person (p), organization (o), location (l), or none (n). The labels are encoded as 'p', 'o', 'l' and 'n' respectively. Assume only these 4 types of NER tags (gold standard labels) are possible. A RNN is used for this classification task. Consider the following input sentence and the gold standard labels that are used to train the model:

Gates the CEO of Microsoft came to visit Delhi.

p n n n o n n n l

Assume only the words in the above sentence are present in your vocabulary. Also, assume that at each time step, one word is fed to the RNN. The output probabilities of the softmax at time step t are $\hat{y}(t) = [\hat{y}_p(t), \hat{y}_o(t), \hat{y}_l(t), \hat{y}_n(t)]$ where $t \in [1,9]$.

(i) What will be the loss *only* at time step t=5?
(ii) What will be the total loss over the entire sentence?
(iii) If you use a bidirectional RNN for this task (instead of a vanilla RNN), how many times will Softmax be called?
(iv) What differences would you make if, instead of NER, the task was to predict the sentiment of a sentence, where 3 sentiments (positive, negative, neutral) are possible?

(b) Suppose we are performing NER on a real-life dataset with a vocabulary size of 8,192 and 4 NER classes. We employ a WordEmbedding + vanilla RNN + Linear + Softmax architecture for this purpose. We set the word embedding dimension to 128, and RNN hidden embedding dimension to 64. Also assume that both the RNN and linear modules are single-layer and are accompanied by the bias parameters. What are the total number of learnable parameters in this architecture?

Question 5 (Seq2seq, cross attention) [(3 + 2) + 5 = 10]

(a) You are using an RNN-based sequence-to-sequence framework with simple dot-product attention. Suppose that the encoder-side hidden state representations are [0,1], [1,1], [1,0], [2,1] at different time steps. If the decoder hidden state at a given time is [1,-2], what will be the attention-weighted encoder representation at this time step? *In terms of alignment with the decoder hidden state*, how can we say that this attention-weighted representation is better than the *mean* of all encoder hidden state representations? Justify numerically.

(b) You have a *bi-gram* language model that has been trained on the following probability transition matrix (cell $[i, j]$ is probability of transition from i to j; A, B and C are tokens).

	<START>	*A*	*B*	*C*	*<END>*
<START>	0	0.4	0.2	0.3	0.1
A	0	0.3	0.35	0.1	0.25
B	0	0.1	0.15	0.5	0.25
C	0	0.3	0.05	0.2	0.45
<END>	0	0	0	0	0

During decoding, you start with the <START> token at time t = 0. Perform a beam search (beam width = 2) up to three time steps (t = 1, 2, 3). Show all steps of the calculation. You must also show all branches of the calculation. If some branch terminates (<END> generated), continue with the other branches. What are the terminated strings obtained within t = 3? What are the scores (sum of log probabilities) for these strings?

Question 6 (Transformers, self attention) [10]

Consider a simplified Transformer encoder module with a single self-attention layer. Assume the model accepts only up to **4** input tokens, and the self-attention layer contains only **1** attention head. Calculate the internal representation $\mathbf{Z}_0$ and $\mathbf{Z}_1$ respectively for each word in the input sentence "**hello world**", after passing the sentence through the self-attention layer of the Transformer encoder module.

The initial word embedding of '**hello**' is [1, -1, 1], and '**world**' is [0, 1, 1]. The model concatenates a 2-bit positional encoding (00/01/10/11) at the end of each input embedding. The Query, Key and Value matrices for the attention head are given below, along with the output weight matrix.

Instead of the softmax operation, use the following scoring function: $s(x_i) = \dfrac{|x_i|}{\sum_{j=1}^{n} |x_j|}$

Show all the intermediate values up to 3 decimal places.

$W^Q = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$	$W^K = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & -1 & 0 \end{bmatrix}$	$W^V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$	$W^O = \begin{bmatrix} -1 & 0 \\ 1 & 1 \\ 0 & -1 \\ 0 & -1 \end{bmatrix}$